ChatGPT

Modelos grandes que pueden ejecutarse localmente con VRAM limitada

Resumen ejecutivo

Sí es posible ejecutar modelos bastante más grandes que los de 0,8B–4B que estabas considerando, incluso en una GPU con poca VRAM, siempre que aceptes **cuantización, descarga parcial a RAM/CPU y, en casos extremos, velocidades muy bajas**. `llama.cpp`, por ejemplo, soporta explícitamente inferencia híbrida CPU+GPU para modelos que superan la VRAM disponible; Hugging Face Accelerate puede repartir automáticamente pesos entre GPU, RAM y, como último recurso, disco. [1]

Como en este encargo no has fijado las especificaciones exactas del PC, **no asumiré una GPU concreta**. Analizaré cinco escalones: **4, 8, 12, 24 y 48+ GB de VRAM**. Además, asumiré inicialmente contexto corto de unos **4K tokens, batch 1 y una sola sesión**. Esto es importante: un modelo que cabe con 4K puede dejar de caber con 32K, 128K o más debido a la caché KV y otras estructuras de inferencia. Accelerate también advierte que una GPU no puede utilizar literalmente toda su VRAM para los pesos: la inicialización de CUDA puede consumir aproximadamente 1–2 GB por sí sola. [2]

Mi conclusión práctica es la siguiente:

VRAM	Tamaño que considero razonable	Tamaño que puede ejecutarse forzándolo
4 GB	**8B–9B Q4** con CPU offload	12B–14B cómodamente con RAM suficiente; 24B–31B muy lento; 70B técnicamente posible con muchísima RAM
8 GB	**8B–9B Q4/Q5**, algunos 12B Q4	14B, 24B y 27B con offload
12 GB	**12B–14B Q4/Q5/Q6**	24B–31B Q4 con CPU offload
24 GB	**24B–31B Q4/Q5**, 8B–14B con cuantización alta	**70B Q4 con offload parcial**
48+ GB	**70B Q4**; modelos 24B–31B en Q8/alta precisión	70B Q5/Q6 al aumentar a 56–64+ GB

Los candidatos que priorizaría son **Qwen3-8B**, **Qwen3.5-9B**, **Gemma 3 12B**, **Ministral 3 14B**, **Mistral Small 3.2 24B**, **Qwen3.8-27B** y **Llama 3.3 70B**. Son un buen recorrido desde un “grande pequeño” de 8B hasta un 70B que sirve para explorar el límite de lo que se puede hacer mediante offload.

Para un PC realmente limitado, mi primera elección sería **Qwen3-8B Q4_K_M/Q5_K_M**. Para subir claramente de capacidad, **Ministral 3 14B Q4_K_M** es especialmente interesante: Mistral lo diseñó para despliegue local/edge y afirma que el 14B puede caber en **24 GB de VRAM en FP8 y menos si se cuantiza más**. [3] En una máquina con 24 GB, **Qwen3.8-27B Q4/Q5** se convierte en una opción mucho más ambiciosa. Con 48 GB, un **Llama 3.3 70B Q4** entra ya en territorio razonablemente realista.

Supuestos y metodología

Aquí utilizo "modelo grande" siguiendo **tu criterio de ≥8B y ≤70B parámetros**. No existe una frontera universal de la industria que diga que 8B ya sea necesariamente un "LLM grande"; simplemente es una clasificación útil para esta comparación.

Los parámetros oficiales se extraen de las fichas de los desarrolladores. Qwen declara **8,2B** para Qwen3-8B y **9B** para Qwen3.5-9B. Este último incluye además un encoder de visión. [4] Google/Hugging Face registra Gemma 3 12B como un modelo de **12B parámetros**. [5] Mistral especifica que Ministral 3 14B combina aproximadamente **13,5B del modelo lingüístico + 0,4B del encoder visual**, y su ficha lo registra como 14B. [3] Mistral Small 3.2 figura como **24B**. [6] Qwen3.8-27B declara **27B parámetros del componente lingüístico** más encoder visual. [7] Llama 3.3 se comercializa como "70B", aunque la metadata de safetensors de Hugging Face contabiliza aproximadamente **71B parámetros**; para las estimaciones mantengo la denominación nominal 70B. [8]

Para calcular memoria hay que separar dos cosas:

Memoria de pesos ≠ VRAM total necesaria para ejecutar el modelo.

Una aproximación útil es:

$$\text{GiB de pesos} \approx \frac{\text{parámetros} \times \text{bits efectivos por peso}}{8 \times 2^{30}}$$

Los formatos GGUF de `llama.cpp` incorporan escalas, metadatos y, en las variantes K, mezclas de cuantización entre tensores. Por ello `Q4_K_M` no ocupa exactamente "4 bits × número de parámetros". En las tablas utilizo como **estimación de planificación**, no como especificación contractual de cada fichero:

Formato	Supuesto aproximado usado
FP16/BF16	16 bits/peso
Q8_0	~8,5 bits/peso
Q6_K	~6,56 bits/peso
Q5_K_M	~5,7–6,0 bits/peso
Q4_0 / Q4_K	~4,5–5,0 bits/peso

`llama.cpp` documenta soporte de cuantizaciones de 1,5 a 8 bits y, de especial interés aquí, la posibilidad de repartir el modelo entre CPU y GPU cuando excede la VRAM. [9]

A la memoria de pesos hay que sumarle **runtime CUDA, caché KV, activaciones, buffers, contexto y eventualmente encoder/proyector de visión**. Por eso las cifras siguientes no deben interpretarse como "si el fichero mide 7 GB, una GPU de 8 GB seguro que lo ejecuta". Accelerate explica justamente que el dispositivo necesita memoria adicional y que puede ser necesario restringir explícitamente la memoria utilizable mediante `max_memory`. [2]

En cuanto a las especificaciones desconocidas de tu PC, mis supuestos son:

Componente desconocido	Efecto
VRAM	Determina cuántos pesos pueden permanecer en la GPU y es el factor principal de velocidad
RAM	Determina cuánto modelo puede descargarse a CPU sin recurrir a SSD/pagefile
CPU	Con mucho offload, pasa a ser determinante; ancho de banda de RAM importa muchísimo
PCIe	Cuanto más se transfieran pesos CPU↔GPU, más importante resulta
SSD/NVMe	Solo debería entrar en juego como último nivel de offload
Sistema operativo	`llama.cpp` funciona bien en Linux y Windows; bitsandbytes también documenta soporte de Windows/Linux en hardware compatible. [10]
Contexto	Más contexto = mayor caché KV = menos memoria disponible para pesos
Multimodalidad	Procesar imágenes/vídeo añade memoria sobre el cálculo puramente textual

Para interpretar "rendimiento esperado" utilizaré tres niveles. **Moderado** significa que la mayoría o todos los pesos están en GPU y el uso interactivo debería ser razonable para la clase de hardware. **Lento** significa offload significativo CPU↔GPU. **Muy lento** significa que gran parte del modelo se está procesando desde RAM/CPU o, peor todavía, desde disco. No doy tokens/s numéricos porque sin conocer GPU, CPU, velocidad de RAM, PCIe y backend sería falsa precisión.

Modelos candidatos y memoria

Esta primera tabla representa aproximadamente **la memoria de los pesos**, no el consumo total del proceso. En `Q4` doy un intervalo que engloba aproximadamente Q4_0 y Q4_K habituales; el fichero GGUF concreto puede desviarse.

Modelo y fuente primaria	Parámetros	FP16 pesos	Q8_0	Q6_K	Q5_K_M	Q4_0 / Q4_K	Mínimo práctico Q4 sin offload*
Qwen3-8B [11]	**8,2B**	~15,3 GiB	~8,1	~6,3	~5,4–5,7	~4,3–4,8	**~6 GB**
Qwen3.5-9B [12]	**9B + visión**	~16,8 GiB†	~8,9†	~6,9†	~6,0–6,3†	~4,7–5,2†	**~7 GB**
Gemma 3 12B IT [5]	**12B**	~22,4 GiB	~11,9	~9,2	~8,0–8,4	~6,3–7,0	**~9 GB**
Ministral 3 14B Instruct [3]	**~14B total**	~26,1 GiB	~13,9	~10,7	~9,3–9,8	~7,3–8,1	**~10–11 GB**

Modelo y fuente primaria	Parámetros	FP16 pesos	Q8_0	Q6_K	Q5_K_M	Q4_0 / Q4_K	Mínimo práctico Q4 sin offload*
Mistral Small 3.2 24B [13]	**24B**	~44,7 GiB de pesos; **~55 GB reales GPU** según Mistral	~23,7	~18,3	~15,9–16,8	~12,6–14,0	**~16–17 GB**
Qwen3.8-27B [7]	**27B + visión**	~50,3 GiB†	~26,7†	~20,6†	~17,9–18,9†	~14,1–15,7†	**~18–20 GB**
Llama 3.3 70B Instruct [14]	**70B nominal / ~71B metadata**	~130–132 GiB	~69–70	~53–54	~46–50	~37–41	**~43–46 GB**

* “Mínimo práctico Q4” presupone contexto corto (~4K), batch 1 y margen básico para el runtime; **no garantiza** que una configuración multimodal o de contexto largo entre.

† Para Qwen3.5 y Qwen3.8, Qwen identifica esas cantidades como parámetros del **Language Model** y declara por separado que la arquitectura contiene encoder visual. Por tanto, calcular únicamente `9B × bits` o `27B × bits` puede subestimar ligeramente el checkpoint multimodal completo. [15]

La diferencia entre “pesos FP16” y “VRAM real” se ve de forma especialmente clara en **Mistral Small 3.2 24B**: el cálculo puramente matemático de 24.000 millones de pesos FP16 da unos 44,7 GiB, pero la propia ficha de Mistral indica aproximadamente **55 GB de GPU RAM para ejecutarlo en BF16/FP16**. [16] Eso explica por qué es peligroso calcular compatibilidad mirando únicamente el tamaño del fichero.

Hay además un candidato especialmente interesante alrededor de este rango: **Gemma 4**, la generación actual de Google en 2026, dispone oficialmente de tamaños 12B, 26B y 31B; Google posiciona los 12B/26B/31B como modelos orientados a “personal computers”. [17] Un Gemma 4 31B denso tendría, por pura escala, aproximadamente **16–18 GiB de pesos Q4**, por lo que conceptualmente pertenece a la misma clase de hardware que Qwen3.8-27B: **24 GB de VRAM es el escalón atractivo**, mientras que 12 GB requeriría CPU offload. Esa última cifra es una estimación de memoria, no una cifra oficial de Google.

Viabilidad por nivel de VRAM

La tabla siguiente es la que utilizaría para decidir qué descargar. “Sin offload” significa que los pesos y el margen de inferencia caben esencialmente en la GPU. “Offload” significa que parte de los pesos permanecen en RAM/CPU y se procesan o transfieren según el backend. `llama.cpp` está específicamente diseñado para inferencia híbrida CPU+GPU; Hugging Face Accelerate puede llenar primero la GPU, después la RAM y finalmente recurrir a almacenamiento si no queda otra alternativa. [1]

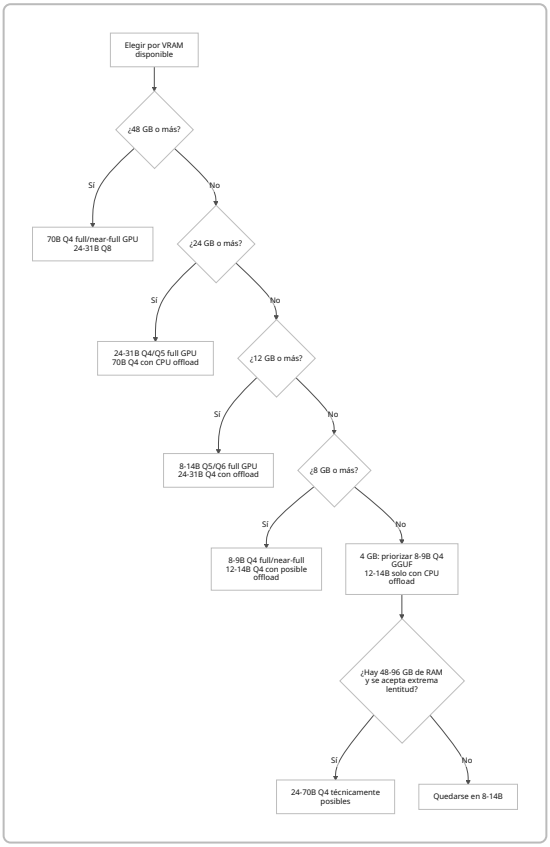
Modelo	**4 GB VRAM**	**8 GB**	**12 GB**	**24 GB**	**48+ GB**
Qwen3-8B	**Sí, offload.** Q4_K_M, `llama.cpp`. **Lento**	**Sí.** Q4 full/ near-full. **Moderado**	**Sí.** Q5/Q6/ Q8. **Moderado**	FP16 o Q8 holgado. **Moderado**	FP16 trivial
Qwen3.5-9B	**Sí, offload.** Q4. **Lento**	**Sí.** Q4; contexto corto. **Moderado**	**Sí.** Q6/Q8. **Moderado**	FP16 con margen razonable. **Moderado**	FP16 holgado
Gemma 3 12B	**Sí, mucho offload.** Q4. **Muy lento/ lento**	**Sí, pero justo.** Q4 + posible offload. **Lento**	**Sí.** Q4/Q5/ Q6. **Moderado**	Q8/Q6; FP16 demasiado justo para recomendarlo	FP16
Ministral 3 14B	**Sí, mucho offload.** Q4. **Muy lento**	Q4 + pequeño/ moderado offload. **Lento**	**Sí.** Q4/Q5. **Moderado**	Q8/FP8. **Moderado**	FP16/BF16
Mistral Small 24B	Técnicamente Q4 + fuerte CPU offload. **Muy lento**	Q4 + fuerte offload. **Muy lento**	Q4 + offload. **Lento**	**Q4/Q5/Q6 full GPU. Moderado**	48 GB: Q8; para FP16 Mistral indica ~55 GB
Qwen3.8-27B	Técnicamente Q4 + mucho offload. **Muy lento**	Q4 + mucho offload. **Muy lento**	Q4 + offload. **Lento**	**Q4/Q5/Q6. Moderado**	Q8; FP16 requiere más de 48 GB
Llama 3.3 70B	Posible solo con enorme RAM. Q4, casi todo CPU. **Muy lento**	Igual; algo más acelerado. **Muy lento**	Q4 con fuerte offload. **Muy lento**	Q4 con offload parcial importante. **Lento/muy lento**	**48 GB: Q4 full/near-full.** 64 GB: Q5/ Q6; ~80 GB+: Q8

En RAM del sistema, mis objetivos conservadores serían aproximadamente estos:

Modelo	RAM aconsejable cuando hay CPU offload
8B–9B Q4	**16 GB mínimo práctico; 32 GB preferible**
12B–14B Q4	**32 GB**
24B–27B Q4	**32 GB puede funcionar en algunos casos; 48–64 GB mucho más cómodo**
~31B Q4	**48–64 GB**
70B Q4	**64 GB es muy justo; 80–96 GB preferible**

Estas últimas son recomendaciones de **headroom**, no límites arquitectónicos. La RAM necesaria disminuye cuanto mayor porcentaje del modelo logres dejar permanentemente en VRAM. Por ejemplo, un 70B Q4 con ~40 GB de pesos no exige que los ~40 GB residan simultáneamente en RAM si una fracción grande está cargada en una GPU de 24 GB; un sistema con GPU de 4 GB, en cambio, tendrá que sostener prácticamente el modelo completo fuera de la GPU.

El criterio de selección puede resumirse así:



Este diagrama presupone contexto corto. Si vas a usar decenas o cientos de miles de tokens, hay que **retroceder al menos un escalón de exigencia** o reservar mucho más espacio para KV cache. Qwen3.5-9B, por ejemplo, anuncia 262.144 tokens nativos, pero que el modelo *admita* ese contexto no significa que tu GPU pueda mantenerlo localmente con el mismo presupuesto de memoria que una conversación de 4K. [12] Qwen3.8-27B también anuncia 262.144 tokens nativos, extensibles hasta un millón, lo que debe entenderse como capacidad arquitectónica, no como promesa de ejecución en una GPU doméstica de poca VRAM. [7]

Estrategias de ejecución y rendimiento

Para **4–8 GB de VRAM**, mi estrategia preferida sería **GGUF + llama.cpp**. Es justamente el escenario para el que resulta atractiva su inferencia híbrida: mantienes en GPU tantas capas como entren y dejas el resto en memoria principal. El proyecto soporta CPU, CUDA y otros backends, así como cuantizaciones 4/5/6/8-bit. [9]

Eso hace que el ranking práctico para una GPU muy pequeña sea aproximadamente:

Qwen3-8B Q4_K_M → Qwen3.5-9B Q4 → Gemma 3 12B Q4 → Ministral 3 14B Q4 → 24B/27B Q4 → 70B Q4.

La capacidad aumenta hacia la derecha, pero el coste computacional y el porcentaje que tiene que quedar en CPU también.

En **12 GB**, los 12B–14B son el "sweet spot": Q4 y Q5 deberían dejar margen suficiente para caché y buffers, mientras que Q6 puede ser viable con contexto corto. En **24 GB**, 24B–27B Q4/Q5 pasan a ser realmente atractivos: ya no dependes necesariamente de transferencia CPU↔GPU y puedes aprovechar un modelo sensiblemente mayor.

Para **Transformers + bitsandbytes**, hay que distinguir sus formatos de GGUF. `load_in_8bit=True` y `load_in_4bit=True` usan los kernels/formas de cuantización de bitsandbytes; un modelo "4-bit bitsandbytes" **no es Q4_K_M**. Hugging Face documenta que la cuantización de 8 bits reduce aproximadamente a la mitad la memoria de pesos y que `device_map="auto"` puede distribuir el modelo entre dispositivos. [18]

Hay una salvedad crucial con **CPU offload de bitsandbytes INT8**: los pesos que terminan despachados a CPU se conservan en **FP32**, según la propia documentación. Esto significa que es una opción estupenda cuando solo necesitas sacar una fracción moderada del modelo de GPU, pero puede consumir muchísima RAM si intentas descargar casi todo un 24B o 70B. [18] Para un 70B sobre una GPU de 4–12 GB, GGUF/llama.cpp suele ser conceptualmente mucho más apropiado.

Hugging Face Accelerate ofrece otra modalidad. Con `device_map="auto"` llena primero las GPU, luego la RAM y, si tampoco basta, puede recurrir a disco mediante memory mapping; las capas se desplazan hacia el dispositivo de ejecución cuando son necesarias. [19] Esto demuestra que "no cabe en VRAM" no equivale necesariamente a "no puede ejecutarse". Pero Hugging Face advierte de que el offload a almacenamiento puede ser muy lento, especialmente si no se dispone de un SSD NVMe rápido. [19]

vLLM tiene su propio `--cpu-offload-gb`. La documentación lo describe literalmente como una forma de proporcionar "VRAM virtual" adicional: por ejemplo, una GPU de 24 GB más 10 GB de CPU offload puede tratarse conceptualmente como si hubiese más espacio para pesos. La contrapartida es que esos pesos deben transferirse CPU↔GPU durante cada pasada, por lo que exige una interconexión rápida para no penalizar muchísimo el rendimiento. [20] Por eso yo reservaría vLLM principalmente para configuraciones en las que **la mayor parte del modelo ya cabe en GPU** y buscas serving/concurrencia, no como primera opción para hacer sobrevivir un 70B en una GPU de 4 GB.

`--swap-space` de vLLM tampoco debe confundirse con `--cpu-offload-gb`: el primero tiene otros usos de memoria temporal; para pesos del modelo la opción relevante es CPU offload. Además, la documentación de troubleshooting de vLLM advierte que cargas grandes pueden provocar fuerte consumo de RAM y swapping del sistema, ralentizando el sistema operativo. [21]

GPTQ tiene sentido cuando quieres un modelo 3/4-bit orientado principalmente a GPU. Hugging Face mantiene soporte GPTQ mediante **GPT-QModel** y señala que AutoGPTQ ya no es el backend soportado en Transformers. [22] Es especialmente interesante en 8–24 GB cuando dispones de un checkpoint GPTQ ya preparado y la cuantización cabe esencialmente en GPU. Para un escenario de offload agresivo CPU/GPU, GGUF suele ofrecer una operativa más sencilla.

Así resumiría los runtimes:

Situación	Runtime/formato que priorizaría
GPU **4 GB**	**llama.cpp + GGUF Q4_K_M + CPU offload**
GPU **8 GB**, 8–9B	llama.cpp Q4/Q5; GPTQ/AWQ/BNB4 si cabe en GPU
GPU **12 GB**, 12–14B	llama.cpp Q4/Q5/Q6 o cuantización GPU 4-bit
GPU **24 GB**, 24–31B	llama.cpp Q4/Q5 o vLLM/GPTQ/AWQ si el checkpoint cabe
GPU **24 GB**, 70B	llama.cpp Q4 + mucho RAM/CPU offload
GPU **48 GB**, 70B	llama.cpp/vLLM Q4 según uso; ya puede ser full-GPU
Varias GPU	vLLM tensor parallel o sharding/`device_map`
VRAM + RAM insuficientes	disk offload solamente como **último recurso**

Para cada candidato, mi combinación preferida sería:

Modelo	Estrategia recomendada
Qwen3-8B	GGUF Q5_K_M/Q6 si cabe; Q4_K_M para 4–8 GB; bitsandbytes/GPTQ para GPU
Qwen3.5-9B	Q4/Q5 local; Transformers/vLLM son oficialmente compatibles según Qwen. [23]
Gemma 3 12B	Q4_K_M en 8–12 GB; Q5/Q6 en 12–24 GB
Ministral 3 14B	Q4/Q5 local; FP8/Q8 en 24 GB. Mistral lo diseña explícitamente para edge/local. [3]
Mistral Small 24B	Q4_K_M/Q5_K_M con 24 GB; vLLM si tienes memoria suficiente. Mistral recomienda vLLM para el checkpoint oficial. [6]
Qwen3.8-27B	24 GB: Q4/Q5; 12 GB: Q4 + offload. La ficha oficial admite Transformers, vLLM y SGLang y enlaza cuantizaciones para apps locales. [7]
Llama 3.3 70B	Q4 GGUF + llama.cpp para consumidor; full-GPU a partir de ~48 GB Q4; la ficha oficial ofrece Transformers/vLLM y enlaza cuantizaciones locales. [24]

Ejemplos de carga en Linux y Windows

El primer ejemplo es el que elegiría para un PC de especificaciones modestas: **Qwen3-8B Q4_K_M con llama.cpp y offload parcial**.

El instalador oficial actual de `llama.cpp` proporciona comandos tanto para shell como PowerShell. [9]

En Linux:

```
curl -LsSf https://llama.app/install.sh | sh
```

En Windows PowerShell:

```
irm https://llama.app/install.ps1 | iex
```

Suponiendo que has descargado un GGUF Q4_K_M compatible:

```
# Linux
llama cli \
  -m ~/models/Qwen3-8B-Q4_K_M.gguf \
  -ngl 20 \
  -c 4096 \
  -p "Explica las diferencias entre TCP y UDP."
```

En Windows:

```
llama cli `
  -m "C:\Models\Qwen3-8B-Q4_K_M.gguf" `
  -ngl 20 `
  -c 4096 `
  -p "Explica las diferencias entre TCP y UDP."
```

`-ngl` / `--n-gpu-layers` controla cuántas capas intentas mantener en la GPU. Si aparece OOM, se reduce. Si todavía queda VRAM libre, se aumenta. El objetivo en una GPU pequeña es encontrar el mayor valor que entre de forma estable. El soporte oficial de `llama.cpp` para inferencia híbrida CPU+GPU es precisamente lo que permite esta estrategia. [9]

En una hipotética GPU de 4 GB empezaría conservadoramente, por ejemplo:

```
llama cli \
  -m ./Qwen3-8B-Q4_K_M.gguf \
  -ngl 10 \
  -c 4096 \
  -p "Hola. Resume este texto:"
```

y subiría `-ngl` hasta acercarme a la VRAM disponible sin alcanzar el límite.

Para **Llama 3.3 70B Q4**, el mecanismo sería conceptualmente el mismo:

```
llama cli \
  -m ./Llama-3.3-70B-Instruct-Q4_K_M.gguf \
  -ngl 20 \
  -c 4096 \
  -p "Analiza este problema paso a paso."
```

Aquí la diferencia es que el fichero cuantizado ronda decenas de GiB, por lo que necesitarás una cantidad considerable de RAM. Con una GPU de 4–12 GB la mayor parte de la inferencia permanecerá

fuera de GPU y la experiencia será **muy lenta**; con 24 GB puede acelerarse una fracción mucho mayor; con ~48 GB un Q4 pasa a poder residir aproximadamente entero en GPU.

Un segundo enfoque es **Transformers + bitsandbytes**. Hugging Face indica la instalación de `transformers`, `accelerate` y `bitsandbytes`, y soporta carga cuantizada junto con `device_map`. [18]

```
python -m pip install -U transformers accelerate bitsandbytes
```

Ejemplo de **Qwen3-8B INT8 con descarga parcial a CPU**:

```
from transformers import (
    AutoModelForCausalLM,
    AutoTokenizer,
    BitsAndBytesConfig,
)

MODEL_ID = "Qwen/Qwen3-8B"

quantization_config = BitsAndBytesConfig(
    load_in_8bit=True,
    llm_int8_enable_fp32_cpu_offload=True,
)

tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)

model = AutoModelForCausalLM.from_pretrained(
    MODEL_ID,
    quantization_config=quantization_config,
    device_map="auto",
    # Ejemplo para una GPU nominal de 8 GB:
    # dejamos margen para CUDA, caché y generación.
    max_memory={
        0: "7GiB",
        "cpu": "28GiB",
    },
    dtype="auto",
)

messages = [
    {"role": "user", "content": "Explica qué es una tabla hash con un
ejemplo."}
]

inputs = tokenizer.apply_chat_template(
    messages,
    add_generation_prompt=True,
    return_tensors="pt",
).to(model.device)
```

```
outputs = model.generate(
    inputs,
    max_new_tokens=256,
    do_sample=False,
)

print(
    tokenizer.decode(
        outputs[0][inputs.shape[-1]:],
        skip_special_tokens=True,
    )
)
```

La advertencia aquí es importante: el mecanismo INT8 de bitsandbytes mantiene en FP32 las partes descargadas a CPU, por lo que **no configuraría un 70B así en una máquina con RAM limitada**. [18]

Con Accelerate puro, `device_map="auto"` puede continuar desde GPU hacia RAM e incluso disco si es imprescindible. Hugging Face documenta que este reparto es automático y que las capas offloaded se desplazan para el cálculo. [19]

Para vLLM, en Linux y en una configuración con suficiente RAM podría usarse el mecanismo de CPU offload de esta forma:

```
vllm serve Qwen/Qwen3-8B \
  --cpu-offload-gb 10 \
  --gpu-memory-utilization 0.85 \
  --max-model-len 4096
```

El significado de `--cpu-offload-gb` está documentado por vLLM como una extensión de la capacidad efectiva para pesos mediante memoria de CPU, con la contrapartida de transferencias CPU↔GPU en cada forward pass. [20]

Para una máquina doméstica con 4–8 GB no lo elegiría como primer experimento; empezaría por GGUF/llama.cpp.

Riesgos, compromisos y recomendación final

El primer riesgo es **OOM**. Que los pesos calculados sean menores que la VRAM anunciada no significa que el modelo vaya a cargar: CUDA, caché KV, buffers y activaciones necesitan memoria adicional. Hugging Face señala incluso que una primera asignación CUDA puede consumir del orden de 1–2 GB antes de contar el propio modelo. [2] Por eso no recomiendo un fichero de 7,8 GB para una GPU de 8 GB y esperar que funcione sin offload.

El segundo riesgo es **confundir RAM con VRAM**. Un modelo puede ejecutarse gracias a 32, 64 o 96 GB de RAM aunque solo tengas 4–12 GB de VRAM, pero eso no convierte la RAM en una GPU lenta equivalente. `llama.cpp` puede repartir CPU/GPU, y vLLM puede utilizar CPU offload, pero las transferencias y el menor ancho de banda reducen mucho el rendimiento. [25]

El tercer riesgo es **swap/pagefile**. Una cosa es descargar pesos deliberadamente a RAM; otra, permitir que el sistema operativo empiece a paginar RAM hacia SSD. Accelerate permite incluso offload explícito a disco, pero advierte de penalizaciones muy importantes si el almacenamiento no es suficientemente rápido. [19] vLLM también advierte de situaciones donde un gran consumo de memoria provoca swapping y ralentización del sistema. [21] Un NVMe rápido puede hacer que algo técnicamente arranque; no hará que se comporte como si tuvieses la RAM que te falta.

El cuarto compromiso es **cuantización frente a calidad**. Cuanto menos bits utilices, menor memoria y ancho de banda necesitas, pero mayor es la perturbación de los pesos. GPTQ nació precisamente como una técnica de cuantización post-entrenamiento a pocos bits para hacer manejables modelos grandes; su objetivo es reducir el error respecto al modelo de precisión completa, pero no significa que toda cuantización sea sin pérdida. [26] Para una máquina local yo consideraría **Q4_K_M el suelo práctico inicial**, **Q5_K_M el equilibrio preferido cuando hay memoria**, Q6_K cuando queda margen y Q8_0 cuando prácticamente quieres conservar la calidad del checkpoint a costa de casi duplicar el peso respecto a Q4.

El quinto compromiso es el **contexto**. No escogería el tamaño del modelo tomando como referencia los contextos máximos anunciados por sus fabricantes. Qwen3.8-27B, por ejemplo, anuncia 262K nativos y hasta 1M extendido, pero en un ordenador doméstico el contexto debería fijarse según la memoria realmente libre. [7] Para probar si un modelo cabe, empezaría con **2K–4K** y solo después incrementaría contexto.

Por tanto, sin saber todavía tu VRAM/RAM exactas, mi jerarquía final sería:

Prioridad	Modelo	Cuantización que probaría primero	Hardware objetivo
Muy alta	**Qwen3-8B**	**Q4_K_M**, luego Q5_K_M	4–8 GB VRAM
Muy alta	**Qwen3.5-9B**	**Q4_K_M/Q5**	4–12 GB
Alta	**Gemma 3 12B**	**Q4_K_M**	8–12 GB
Muy alta	**Ministral 3 14B**	**Q4_K_M/Q5_K_M**	8–12 GB con offload; 12–24 GB ideal
Alta	**Mistral Small 3.2 24B**	**Q4_K_M**	12 GB con offload; **24 GB recomendado**
Muy alta para 24 GB	**Qwen3.8-27B**	**Q4_K_M/Q5_K_M**	**24 GB**; 12 GB con offload
Experimental en PC pequeño	**Llama 3.3 70B**	**Q4_K_M**	24 GB + mucha RAM; **48 GB+ preferible**

Con 4 GB de VRAM, el modelo grande que probaría primero sería **Qwen3-8B Q4_K_M mediante llama.cpp**, haciendo offload de las capas que no entren. Un **9B** sigue siendo perfectamente defendible. Un **12B–14B** también es viable si tienes, idealmente, 32 GB de RAM, pero ya dependerás mucho más del CPU. Un **24B–27B** puede arrancar con suficiente RAM, pero entra claramente en la categoría de "lo ejecuto porque puedo, no porque sea rápido". Un **70B Q4** sería una prueba extrema: con una GPU diminuta necesitarías aproximadamente **64 GB de RAM como mínimo muy ajustado y preferiblemente 80–96 GB**, y aun así gran parte del cálculo quedaría del lado de CPU.

Con 8 GB, 8B–9B Q4 dejan de ser un experimento y pasan a ser una opción sensata; 12B–14B entran mediante Q4 y pequeño offload. **Con 12 GB**, mi candidato favorito sería un **14B Q4_K_M/Q5_K_M**. **Con 24 GB**, el salto más interesante es a **24B–27B Q4/Q5**: ahí Qwen3.8-27B se convierte en una propuesta mucho más atractiva que intentar un 70B fuertemente descargado a CPU. **Con 48 GB**, finalmente, un **70B Q4** se vuelve una opción local seria, mientras que los 24B–31B pueden ejecutarse con cuantizaciones considerablemente menos agresivas.

La regla práctica más importante es ésta: **es preferible un 14B Q5 casi completamente en GPU a un 70B Q4 que pasa la mayor parte del tiempo esperando a CPU/disco**, salvo que tu objetivo sea deliberadamente comprobar cuál es el modelo máximo que la máquina puede arrancar. Para uso diario, la residencia en GPU y el ancho de banda suelen ser tan decisivos como el número bruto de parámetros. `llama.cpp` proporciona precisamente la flexibilidad para encontrar ese punto intermedio capa por capa, mientras que Accelerate, bitsandbytes y vLLM ofrecen alternativas más apropiadas cuando el hardware y el caso de uso se acercan al despliegue predominantemente GPU. [27]

---

[1] [9] [10] [25] [27] GitHub - ggml-org/llama.cpp: LLM inference in C/C++ · GitHub
https://github.com/ggml-org/llama.cpp

[2] [19] Loading big models into memory · Hugging Face
https://huggingface.co/docs/accelerate/concept_guides/big_model_inference

[3] mistralai/Ministral-3-14B-Instruct-2512 · Hugging Face
https://huggingface.co/mistralai/Ministral-3-14B-Instruct-2512

[4] [11] Qwen/Qwen3-8B · Hugging Face
https://huggingface.co/Qwen/Qwen3-8B

[5] google/gemma-3-12b-it · Hugging Face
https://huggingface.co/google/gemma-3-12b-it

[6] [13] [16] mistralai/Mistral-Small-3.2-24B-Instruct-2506 · Hugging Face
https://huggingface.co/mistralai/Mistral-Small-3.2-24B-Instruct-2506

[7] Qwen/Qwen3.8-27B · Hugging Face
https://huggingface.co/Qwen/Qwen3.8-27B

[8] [14] [24] meta-llama/Llama-3.3-70B-Instruct · Hugging Face
https://huggingface.co/meta-llama/Llama-3.3-70B-Instruct

[12] [15] [23] Qwen/Qwen3.5-9B · Hugging Face
https://huggingface.co/Qwen/Qwen3.5-9B

[17] Gemma 4
https://deepmind.google/models/gemma/gemma-4/?utm_source=chatgpt.com

[18] Bitsandbytes · Hugging Face
https://huggingface.co/docs/transformers/quantization/bitsandbytes

[20] Basic — vLLM
https://docs.vllm.ai/en/v0.8.1/getting_started/examples/basic.html?utm_source=chatgpt.com

[21] Troubleshooting
https://docs.vllm.ai/en/latest/usage/troubleshooting/?utm_source=chatgpt.com

[22] GPTQ · Hugging Face
https://huggingface.co/docs/transformers/en/quantization/gptq?utm_source=chatgpt.com

26 GPTQ: Accurate Post-Training Quantization for Generative ...
https://arxiv.org/abs/2210.17323?utm_source=chatgpt.com